



Dokumentace projektu IPP

Interpret jazyka SOL26

Autor: Hana Liškařová (xliskah00)

Varianta: Python + TypeScript

22. dubna 2026

Autor: Hana Liškařová

Login: xliskah00

Obsah

1	Celkový popis návrhu řešení	1
1.1	Orchestration vrstva	1
1.2	Runtime model	1
1.3	Execution vrstva	1
1.4	UML diagram tříd	2
2	Hlavní interní datové struktury	4
2.1	Registry	4
2.2	Runtime třídy a metody	4
2.3	Runtime hodnoty a objekty	4
2.4	Lokální rámce a bloky	5
3	Využití návrhových vzorů a principů OOP	5
3.1	Factory	5
3.2	Strategy	5
3.3	Adapter	5
3.4	Single Responsibility Principle	6
3.5	Liskov Substitution Principle	6
3.6	Zapouzdření, dědičnost, polymorfismus	6
4	Problémy během implementace a jejich řešení	6
4.1	self a super	6
4.2	Bloky	7
4.3	Instanční atributy	7
5	Úpravy ve vylepšené verzi	7
6	Možnosti rozšíření	7
6.1	1. Mechanismus výjimek	7
6.1.1	Zadání	7
6.1.2	Návrh implementace	8
6.2	5. Třídy jako objekty první kategorie	8
6.2.1	Zadání	8
6.2.2	Návrh implementace	8
7	Využití prostředků AI	9

7.1	ChatGPT GPT-5.4 Thinking	9
7.1.1	Rozsah a způsob využití	9
7.2	Přiložené soubory	10
7.3	Junie	11
7.4	Github Copilot GPT 5-mini	11
8	Hlavní využití zdroje	11

1 Celkový popis návrhu řešení

Celý interpret je navržen jako čtyři na sebe navazující vrstvy, z nichž jedna je AST převzaté ze šablony. Ostatní jsou popsány níže.

1.1 Orchestration vrstva

Tato vrstva řídí celý program. Vstupním bodem interpretu je třída `Interpreter`. Ta načte XML soubor, předá jej třídě `XmlValidator` k validaci a ta při chybě vrátí návratový kód 42. Z validovaného XML potom vytvoří objekt `Program`, tedy reprezentaci AST. Samotné spuštění programu řídí `ProgramRunner`: nejdříve spustí `ProgramValidator` pro nalezení statických sémantických chyb v AST. Poté připraví pomocné části potřebné pro vyhodnocování zpráv, konkrétně `AttributeDispatchResolver`, `AttributeAccessor`, `InstanceSendExprEvaluator` a `ClassSendExprEvaluator`. Dále připraví pomocné funkce pro vestavěné metody, které při běhu potřebují vyhodnocují zprávy nad runtime objekty. Tyto funkce následně předá do `RuntimeBuilder`. Ten z AST sestaví objekt `Runtime`, zaregistruje vestavěné i uživatelské prvky a při registraci vestavěných metod k nim tyto pomocné funkce rovnou připojí.

Po vytvoření runtime prostředí se sestaví vykonávací řetězec. Vznikne `ExpressionDispatcher`, nad ním `BlockExecutor` a nad nimi `MethodExecutor`. `MethodExecutor` se potom naváže do `InstanceSendExprEvaluator` a `ClassSendExprEvaluator` a `BlockExecutor` do všech uživatelských metod v runtime. Následně `EntryPointResolver` v připraveném runtime vyhledá vstupní bod programu, tedy objekt třídy `Main` a jeho metodu `run`, kterou spustí `MethodExecutor`.

1.2 Runtime model

Runtime model tvoří běhovou reprezentaci programu připravenou pro samotnou interpretaci. Jeho hlavním objektem je třída `Runtime`, která sdružuje registry tříd a vestavěných prvků, tvorbu runtime hodnot, vstupní a výstupní rozhraní programu. Sestavení této vrstvy zajišťuje `RuntimeBuilder`. Ten do `Runtime` zaregistruje vestavěné třídy, objekty `true`, `false` a `nil`, vestavěné metody a uživatelské třídy. Registrace uživatelských tříd probíhá ve dvou průchodech. V prvním se pro všechny uživatelské třídy vytvoří jejich runtime reprezentace (prázdné instance `RuntimeClass`) a ve druhém se mezi nimi doplní dědičnost a ostatní vazby závislé na existenci dalších tříd. Toto rozdělení je zde proto, aby při doplňování dědičnosti už byly všechny třídy v registru tříd založené a bylo na ně možné odkazovat bez ohledu na pořadí, ve kterém existují ve vstupním programu.

V runtime vrstvě se tak připraví běhové třídy, runtime objekty, reprezentace metod i lokální kontext potřebný pro vykonávací vrstvu. Konkrétní podoba těchto struktur je popsána v sekci 2.

1.3 Execution vrstva

Tato vrstva provádí samotnou interpretaci programu. Její vrchní třídou je `MethodExecutor`, který sjednocuje volání runtime metod. Před každým voláním zkontroluje aritu argumentů, vytvoří `InvocationContext` a předá vykonání samotné metodě. Vzniklý kontext má aktuální objekt a třídu vlastníci právě vykonávanou metodu. Tím má vykonávací vrstva při každém volání k dispozici kontext potřebný pro rozlišení `self` a `super`. `BlockExecutor` vykonává těla metod a bloků jako posloupnost přiřazení a vrací hodnotu posledního z nich. Vyhodnocování výrazů provádí `ExpressionDispatcher`, rozlišuje: proměnnou, literál, blok a zaslání zprávy. U proměnných pracuje s `ScopeFrame` a `InvocationContext`, u literálů

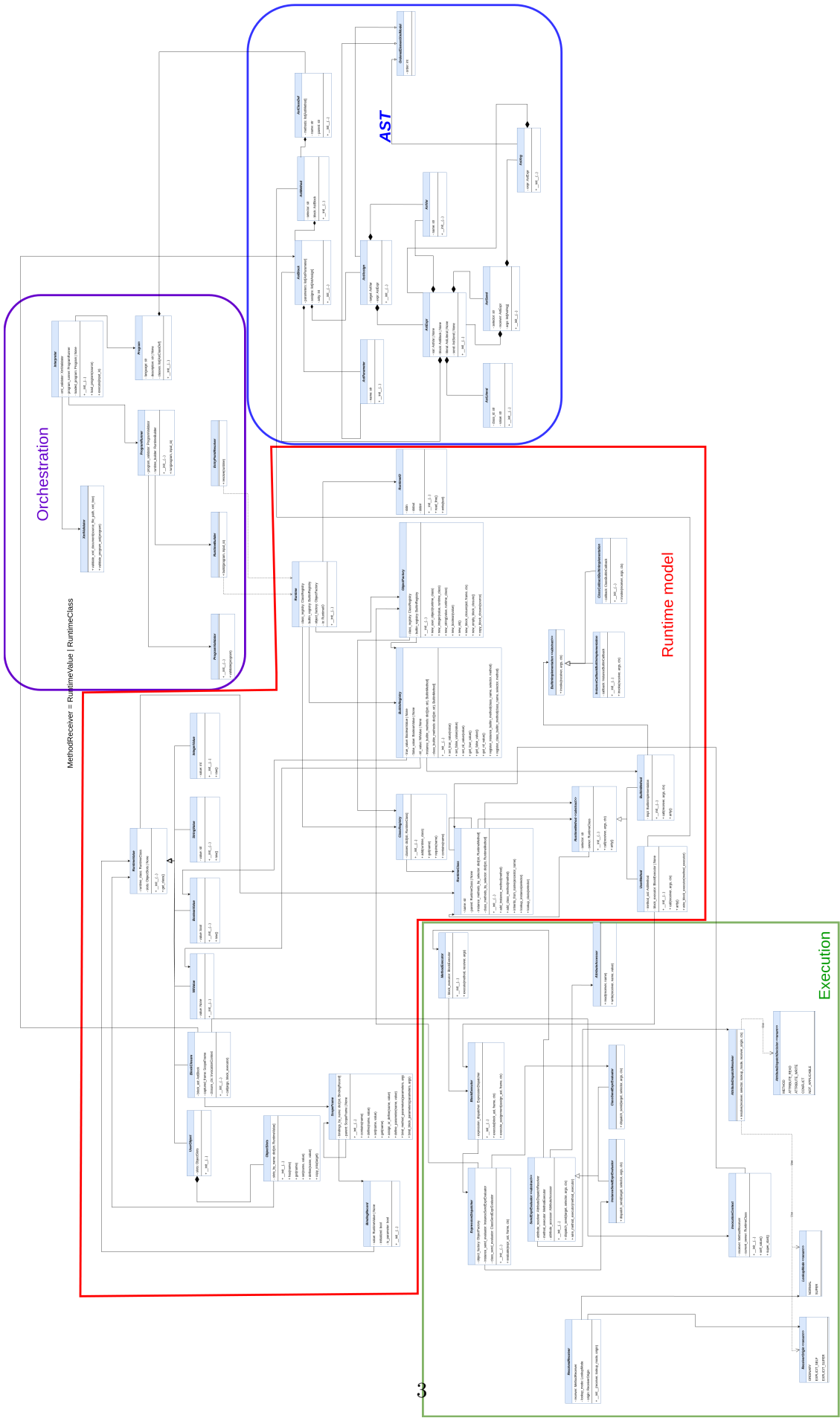
vytváří runtime hodnotu přes `ObjectFactory` a u bloku vytváří nový `BlockClosure` s aktuálním lokálním rámcem a kontextem ve kterém vznikl.

Nejsložitější část této vrstvy představuje vyhodnocování zaslaných zpráv. `ExpressionDispatcher` vyhodnotí objekt, kterému je zpráva zaslána a všechny její argumenty. Samotné vyhodnocení zprávy je rozdělené mezi `InstanceSendExprEvaluator` (řeší zprávy zaslané runtime objektům) a `ClassSendExprEvaluator` (řeší zprávy zaslané třídám). `InstanceSendExprEvaluator` nejprve kontroluje, zda nejde o volání bloku (`value`, `value:`, `value:value:` atd.). Pokud ano, blok vykoná. Jinak přes `AttributeDispatchResolver` rozhodne, zda selektor znamená čtení atributu, zápis atributu, volání metody nebo konflikt mezi atributem a metodou. Čtení a zápis provádí `AttributeAccessor`. Pokud selektor neznamena práci s atributem, interpreter jej vyhodnotí jako běžné volání instanční metody. `ClassSendExprEvaluator` zpracovává zprávy zaslané runtime třídám. Řeší jen vyhledání odpovídající třídní metody v `RuntimeClass` a její vykonání přes `MethodExecutor`.

1.4 UML diagram tříd

Protože je UML diagram tříd příliš veliký a čitelně se nevešel na stránku A4, je zde přidán i odkaz na drawio, v kterém byl vytvořen a lze lépe zobrazit (lze zde i přibližovat a oddalovat). Je úmyslně vizuálně a barevně rozdělen do výše popsaných čtyř sekcí. Třídy obsažené v šabloně jsou zde zakreslené s předponou `Ast`, v šabloně takto pojmenovány nejsou. Je to takto zakreslené schválně, jelikož v kódu jsou třídy z AST vždy importovány a používán s touto předponou pro lepší čitelnost. Dále jsou zde vynechané kvůli čitelnosti některé vztahy - například u `BuiltinRegistry` atributů je pouze jeden vztah na `BooleanValue`.

[Odkaz na UML Class diagram](#)



2 Hlavní interní datové struktury

2.1 Registry

Runtime je hlavním prvkem runtime vrstvy. Drží `ClassRegistry`, `BuiltinRegistry`, `ObjectFactory` a `RuntimeIO`.

`ClassRegistry` ukládá runtime třídy `RuntimeClass` do slovníku `classes` indexovaného jejich jménem. `RuntimeBuilder` odsud doplňuje rodiče tříd, `ExpressionDispatcher` vyhodnocuje literál identifikátoru třídy a `ObjectFactory` zde bere odpovídající `RuntimeClass` při vytváření nového objektu.

`BuiltinRegistry` uchovává objekty `true`, `false` a `nil` v atributech `true_value`, `false_value` a `nil_value`. Dále drží vestavěné metody rozdělené do slovníků `instance_built_in_methods` a `class_built_in_methods` indexovaných pomocí (`class_name`, `selector`) - třída které metoda patří a selektor metody. Samotné vestavěné metody jsou uloženy jako objekty `BuiltinMethod`. `RuntimeBuilder` odsud registruje vestavěné metody ke třídám a `ObjectFactory` bere objekty `true`, `false` a `nil`.

2.2 Runtime třídy a metody

`RuntimeClass` reprezentuje jednu runtime třídu. Uchovává jméno, odkaz na rodiče a metody rozdělené do slovníků `instance_methods_by_selector` a `class_methods_by_selector`, protože interpreter zvlášť vyhledává metody při zaslání zprávy instanci a při zaslání zprávy třídě.

`RuntimeMethod` je společná abstraktní nadtřída runtime metod. Existuje, aby *Execution* vrstva nemusela u metod rozlišovat jestli jde o uživatelskou nebo vestavěnou metodu.

`UserMethod` reprezentuje metodu z AST a drží ji v atributu `method_ast`. Při volání vytvoří nový rámec lokálních vazeb a vykonání deleguje na `BlockExecutor`.

`BuiltinMethod` reprezentuje jednu vestavěnou metodu. Obsahuje objekt typu `BuiltinImplementation`, na který při volání deleguje provedení odpovídajícího chování.

`BuiltinImplementation` obaluje konkrétní chování zapsané jako callback. Callback je použit proto, že vestavěné metody jsou v implementaci funkce, ale runtime model metod pracuje s objekty s rozhraním `invoke(...)`. Třídy `InstanceCallbackBuiltinImplementation` a `ClassCallbackBuiltinImplementation` callback uchovávají jako implementaci vestavěné metody, kontrolují očekávaný typ příjemce zprávy (`RuntimeValue` vs `RuntimeClass`) a po kontrole vyvolají samotné provedení callbacku.

2.3 Runtime hodnoty a objekty

Společným základem všech runtime hodnot je `RuntimeValue`. Uchovává odkaz na svou runtime třídu a u objektů, které mají instanční atributy i `ObjectSlots`. Všechny runtime hodnoty mají tímto společnou reprezentaci a dá se s nimi pracovat stejným způsobem.

Konkrétními potomky `RuntimeValue` jsou `IntegerValue`, `StringValue`, `BooleanValue` a `NilValue`. `IntegerValue`, `StringValue` a `BooleanValue` kromě odkazu na runtime třídu drží i interní hodnotu v atributu `value`. Objekty `true`, `false` a `nil` jsou vytvořeny jen jednou v `RuntimeBuilder` a uloženy v `BuiltinRegistry`.

`UserObject` reprezentuje běžnou instanci uživatelské třídy. Drží svou runtime třídu drží a `ObjectSlots` - instanční atributy. Instanční atributy jsou tedy uloženy v konkrétní instanci.

`ObjectSlots` reprezentuje instanční atributy - ukládá instanční atributy do slovníku `slots_by_name` indexovaného jejich jménem.

2.4 Lokální rámce a bloky

`ScopeFrame` ukládá lokální vazby do slovníku `bindings_by_name` indexovaného jménem a drží odkaz na rodičovský rámec. Díky tomu může interpret při čtení nebo zápisu jména přes rodiče procházet nadřazené prostředí.

`BindingRecord` reprezentuje jednu lokální vazbu uloženou ve `ScopeFrame`. Kromě samotné hodnoty drží i příznaky - zda byla vazba inicializovaná, zda jde o parametr metody nebo bloku.

`BlockClosure` reprezentuje jeden blok spolu s prostředím, ve kterém vznikl. Ukládá jeho AST, zachycený lokální rámec a kontext vzniku. Blok lze tudíž vyhodnotit i později, se zachováním vazeb na okolní prostředí zachycené při jeho vytvoření.

`InvocationContext` reprezentuje kontext právě vykonávané metody. Uchovává aktuální objekt a třídu vlastníci vykonávanou metodu, takže podle něj jde při běhu rozlišit `self` a `super`.

3 Využití návrhových vzorů a principů OOP

Všechny vzory a principy byly přebrány podle [3].

3.1 Factory

Pro vytváření runtime objektů a hodnot je použit vzor **Factory** - třída `ObjectFactory`. Ta vytváří runtime reprezentace `UserObject`, `IntegerValue`, `StringValue` a `BlockClosure`, a vrací sdílené objekty `true`, `false` a `nil` uložené v `BuiltinRegistry`. Důvodem použití je sjednocení veškeré logiky vytváření runtime objektů na jedno místo. Různé objekty potom mají různá pravidla - například u vytvoření nové runtime reprezentace bloku se k novému objektu musí navázat i zachycený rámec + kontext, ale u jiných objektů se jen přidává runtime třída a úložiště instančních atributů. Ve zbytku implementace tedy už není nutné řešit logiku vytváření runtime objektů. Zároveň lze jednoduše rozšířit přidáním nového pravidla pro případný nový objekt.

3.2 Strategy

U vestavěných metod je použit vzor velmi blízký **Strategy**. `BuiltinMethod` neobsahuje přímo implementaci vestavěného chování, ale drží objekt `BuiltinImplementation`. Na ten při volání deleguje provedení přes metodu `invoke`. `BuiltinMethod` je tedy kontext a `BuiltinImplementation` strategie, která určuje konkrétní chování. Vzor *Strategy* není úplně přesně dodržen - strategie v implementaci není vyložena sada samostatných tříd. Je uložena jako sada callbacků, které jsou obalené do tříd `InstanceCallbackBuiltinImplementation` a `ClassCallbackBuiltinImplementation`. Návrhový vzor je zde zaveden, aby mohla existovat společná reprezentace vestavěné metody a její implementace se mohla uchovávat zvlášť. Díky tomu lze jednoduše zaregistrovat nové vestavěné metody - přidá se jen další obalený callback. Zbytek implementace taky může pracovat s `RuntimeMethod` nezávisle na její implementaci.

3.3 Adapter

V implementaci vestavěných metod je také použit vzor **Adapter**, konkrétně ve třídách `InstanceCallbackBuiltinImplementation` a `ClassCallbackBuiltinImplementation`. Dle návrhového vzoru `Client` je `BuiltinMethod`, `Adaptee` je samotný callback,

`InstanceCallbackBuiltinImplementation` a `ClassCallbackBuiltinImplementation` jsou `Adapter` a `Target` je rozhraní `BuiltinImplementation` s metodou `invoke(receiver, args, ctx)`. Adaptery převádějí konkrétní callbacky na rozhraní používané runtime vrstvou. Jsou oddělené pro instanční a třídní vestavěné metody, protože každý před vyvoláním callbacku kontroluje, zda příjemce zprávy odpovídá správné straně. Adapter je použit proto, že callback nemá rozhraní, které očekává runtime vrstva. Díky zajištěnému jednotnému rozhraní lze v `BuiltinMethod` pracovat jednotně s každou implementací.

3.4 Single Responsibility Principle

Jednotlivé části běhu programu jsou jasně rozdělené mezi samostatné třídy podle toho, jaký problém řeší, např: `ProgramRunner` řídí spuštění programu, `RuntimeBuilder` skládá runtime model, `EntryPointResolver` hledá vstupní bod, `MethodExecutor` vykonává metody, `BlockExecutor` vykonává bloky, `ExpressionDispatcher` vyhodnocuje výrazy atd. je díky tomu znažší orientovat se v kódu a dělat změny. U rozšíření se potom dá doplňovat funkcionalita jen do dané části bez ovlivnění ostatních.

3.5 Liskov Substitution Principle

Je využit v hierarchii runtime metod. Primárně u `RuntimeMethod`, ze které dědí `UserMethod` a `BuiltinMethod`. `MethodExecutor` pracuje se společným rozhraním `call(...)` a `arity()`, nerozlišuje kterou z dědicích tříd právě vykonává. To soustřeďuje logiku volání metod na jedno místo a konkrétní rozdíl mezi uživatelskou a vestavěnou metodou zůstává zapouzdřený až v implementaci. Z hlediska rozšiřitelnosti by bylo možné přidat další variantu runtime metody jako nového potomka `RuntimeMethod` bez zásahů do již existující logiky interpretu.

3.6 Zapouzdření, dědičnost, polymorfismus

Zapouzdření - jednotlivé části interpretu si nesou vlastní stav a vlastní operace nad ním. Například `RuntimeClass` si sama drží své metody a lookup nad nimi, `ClassRegistry` si drží registr tříd, `BuiltinRegistry` vestavěné objekty a vestavěné metody atd. **Dědičnost** je v implementaci taky hojně využita (viz diagram tříd). **Polymorfismus** je např. u `MethodExecutor` - pracuje se společným typem `RuntimeMethod` a nerozlišuje, zda právě volá `UserMethod` nebo `BuiltinMethod`. Podobně i pro `RuntimeValue` je zbytek návrhu postavený tak, že pracuje s obecnou runtime hodnotou.

4 Problémy během implementace a jejich řešení

4.1 self a super

Jedním z nejobtížnějších problémů byla implementace `self` a `super`. Konkrétní problém byl reprezentace faktu, že `self` a `super` jsou stejný objekt, který ale má mít odlišné chování. Původní myšlenka byla reprezentovat je jako dva různé objekty - vytvořit `super` jako kopii objektu `self`. Přičemž `super` by pouze hledala metodu v nadtřídě. Zde nastal problém konkrétně v změnách přes `super`. Například zápis atributu přes `super` se zapsal pouze do kopie. Další problém nastal při předání `super` jako argumentu nebo jeho uložení do proměnné. Dál se totiž nešířil původní objekt, ale jeho kopie. Následné operace tudíž běžely nad jiným stavem než měl `self`. Tento problém byl vyřešen přidáním `InvocationContext`. Ten pouze uchovává dva atributy: `receiver` (objekt, na kterém se vykonává

metoda - `self`) a `current_owner` (třída vlastníci vykonávanou metodu). Pro `super` se tedy metody vyhledává v nadtřídě `current_owner`. `ExpressionDispatcher` v případě že `super` je příjemce zprávy nastaví odlišný způsob vyhledávání metody. K tomu slouží konkrétně `LookupMode`. Vyhodnocený příjemce zprávy se potom ukládá jako `ResolvedReceiver`, který nese daný objekt a zvolený způsob vyhledávání. Tím zůstává v obyčejných výrazech stejný objekt jako pro `self`, ale při zaslání zprávy se změní začátek vyhledávání metody.

4.2 Bloky

Hlavní problém u implementace bloků bylo zachycení prostředí v kterém blok vznikl - tedy zachycení lokálních proměných, `self` a `super` atd. Toto bylo vyřešeno zavedením `BlockClosure` v runtime. Při vyhodnocování výrazů se u nalezení bloku ve výrazu blok neprovede. Přes `ObjectFactory` se vytvoří nový objekt `BlockClosure`, který drží informace o AST bloku z kterého vznikl, zachyceném `InvocationContext` (zajišťuje že `self` a `super` má z místa svého vzniku) a rámec prostředí z místa svého vzniku. K aktivaci bloku a jeho provedení nastane až při zaslání zprávy `value, value:, value:value:` atd. Při spuštění se navíc nepoužije přímo zachycený rámec, ale vytvoří se nový `ScopeFrame`, který na něj odkazuje jako na rodiče a do kterého se navážou parametry bloku. Každé spuštění bloku má tedy vlastní lokální prostředí, ale zároveň stále vidí proměnné zachycené z místa svého vzniku.

4.3 Instanční atributy

Hlavním problémem byl fakt, že v SOL26 nejde podle syntaxe oddělit metody od instančních atributů. Volání metody a veškerá práce s instančními atributy je dělaná stejným způsobem pomocí zpráv. Tento problém řeší třída `AttributeDispatchResolver`. Podle typu selektoru rozhoduje zda se jedná o metodu, čtení atributu, zápis atributu nebo konflikt. U bezparametrického selektoru nejprve kontroluje metodu a potom případné čtení atributu. U jednoparametrického selektoru se kontroluje metodu, existující atribut a případně vznik nového atributu. Samotnou práci s atributem potom provádí `AttributeAccessor`, který čte a zapisuje hodnoty v `ObjectSlots`. Pokud se atribut řeší přes `self`, kontroluje se konflikt i proti metodám v aktuální třídě. U `super` se aktuální třída se přeskočí a kontrola je až vůči metodám v nadtřídách.

5 Úpravy ve vylepšené verzi

Jediné zavedené úpravy byly v souboru `Dockerfile`. Hlavní oprava bylo odstranění vložených skriptů přes `COPY «'BASH'»`. Dále také přechod ze `slim` verzí potřebných knihoven na `bookworm-slim`, odstranění ručního nastavování `BASH_ENV` a linkování nástrojů při startu shellu, místo toho přechod na `uv`. Také přidání souborů `ruff`, `mypy`, `prettier` a `eslint`.

6 Možnosti rozšíření

6.1 1. Mechanismus výjimek

6.1.1 Zadání

V tomto rozšíření by blok podporoval zprávu `on:do:`, kde první argument určuje třídu zachytávané výjimky a druhý argument je jednoparametrický catch blok. Třída `Object` by zároveň nově podporovala

zprávu `signal`, která by vyvolala výjimku a spustila *stack unwinding*.

6.1.2 Návrh implementace

Nová zpráva `signal` by se registrovala v `object_builtins.py` mezi *instanční vestavěné metody* třídy `Object`. Vyvolala by výjimku s odkazem na objekt, na kterém byla zpráva `signal` zavolána. Pro ni by bylo nutné vytvořit novou třídu. Zachytávání přes `on:do:` by se doplnilo do `block_builtins.py`. `BlockClosure` už uchovává tělo bloku, zachycený rámec i kontext volání - `catch` blok by se tedy mohl spustit stejně jako normální blok, měl by ale jako parametr objekt výjimky. V *Execution* vrstvě by změny byly minimální: `MethodExecutor`, `BlockExecutor` a `BlockClosure.call` by novou výjimku propouštěly dál a případně do ní při průchodu doplňovaly informace pro zásobník volání. *Stack unwinding* by tak v podstatě vznikl sám při návratu přes už existující volací řetězec. Jediný větší zásah do stávající logiky by byl u třídních literálů. První argument zprávy `on:do:` určuje třídu zachytávané výjimky, ale `ExpressionDispatcher` momentálně umí třídní literál použít jen jako příjemce zprávy, ne jako hodnotu v argumentu. Hlavní nutné změny by tedy byly:

1. vytvoření nové třídy pro reprezentaci výjimky,
2. zaregistrování dvou nových vestavěných zpráv `signal` a `on:do:`,
3. rozšíření `ExpressionDispatcher`, aby umožňoval použít třídní literály jako hodnotu v argumentu.

6.2 5. Třídy jako objekty první kategorie

6.2.1 Zadání

Třídy by se staly objekty první kategorie, tedy instancemi metatřídy `Class`. Podporovaly by zprávy `name` (vrací `String`), `superclass` (vrací objekt třídy), `methods` (vrací kolekci selektorů), `instanceOf:` a `subclassOf:`. Každý objekt by zároveň odpovídal na zprávu `class` a proměnné odkazující na třídy by mohly přijímat zprávy `new` a `from:` jako běžné zprávy.

Součástí rozšíření by byla i podpora třídních atributů, tedy sdíleného měnitelného stavu napříč všemi instancemi dané třídy, inicializovaného před spuštěním `Main»run`, a také uživatelsky definovaných třídních metod volaných zasláním zprávy identifikátoru třídy. Třídní metody by se dědily a v potomcích by je bylo možné překrýt. Jejich vyhledávání v řetězci dědičnosti by fungovalo obdobně jako u instančních metod, ale používala by se pro ně samostatná tabulka metod.

6.2.2 Návrh implementace

Zpracování třídních zpráv a metod a jejich oddělení od instančních metod už v implementaci funguje. Největší změnou by tedy bylo umožnit zacházet s třídou jako s běžnou runtime hodnotou, tedy aby mohla být výsledkem výrazu, uložit se do proměnné nebo předat jako argument. Bylo by nutné upravit `ExpressionDispatcher`, aby uměl třídní literál vyhodnotit i jako hodnotu, ne jen jako příjemce zprávy. Uživatelsky definované třídní metody by se při vytváření runtime modelu ukládaly do třídní části `RuntimeClass`. Třídní atributy by se ukládaly přímo do `RuntimeClass` kde by bylo nutné doplnit samostatné úložiště pro tento stav. Dědění třídních metod by přitom šlo využít téměř beze změny.

Hlavní nutné změny by tedy byly:

1. úprava `ExpressionDispatcher`, aby uměl třídní literál vyhodnotit i jako hodnotu,
2. ukládání uživatelsky definovaných třídních metod do třídní části `RuntimeClass`,
3. doplnění samostatného úložiště pro třídní atributy přímo do `RuntimeClass`.

7 Využití prostředků AI

AI nástroje byly využity jako asistence a generátory testů. Výsledný OOP návrh i implementace je navržena mnou - autorem.

7.1 ChatGPT GPT-5.4 Thinking

Tento LLM byl využíván v webovém provedení. Byl vytvořen projekt, kde bylo AI nástroji dodáno jako projektové zdroje následující:

- Zadání projektu
- Specifikace SOL26
- Veškeré šablony projektu
- Tipy, triky a zpětné vazby k projektům minulých let z [Elearningu](#).

7.1.1 Rozsah a způsob využití

Hlavní oblasti využití byly následující:

- Generování testů
 - AI nástroj podle specifikace SOL26 navrhoval a generoval jednotlivé testy i testovací sady pro interpret i testovací nástroj. Byl používán také při opravách nevalidních testů a při vyhodnocování testovacích reportů.
- Návrh OOP a pracovní plán projektu
 - S AI nástrojem bylo podrobně konzultováno zadání projektu, rozdělení implementace mezi Python a TypeScript i celkový způsob zahájení práce na projektu. S jeho pomocí byla navržena a průběžně upravována prvotní OOP architektura interpretu. AI nástroj sloužil jako pomocník pro zachycení počáteční objektově orientované představy před implementací. Výsledná architektura se liší. Dále byl AI nástroj využit pro návrh postupu vývoje celého projektu a jako průběžná kontrolní pomůcka během implementace. Konečný návrh není dílem AI.
- Chytrý vyhledávač v zadání
 - Byl používán k dohledání konkrétních informací ze zadání a ze specifikace SOL26. Například pravidla pro atributy a dědičnost.
- Specifikace zadání
 - Používán hlavně při interpretaci nejednoznačných nebo velmi detailních částí specifikace. Dotazy na to zda lze použít specifické knihovny a podobné problémy.
- Kontejnerizace a Dockerfile problémy

- Vysvětlení jak sestavit Dockerfile, co způsobuje problémy u build procesu a check prostředí. AI nástroj byl používán pro hledání pravděpodobných příčin chyb, návrh ověřovacích příkazů a interpretaci výstupů z kontejneru.
- Vysvětlení python základů
 - Vysvětlení Python konceptů, například dědičnosti, typů metod, významu `self` atd.
- Šablona dokumentace
 - Vygenerování šablony dokumentace podle dokumentace projektu z jiného předmětu. Pomoc se stylováním dokumentace a s pravopisnými chybami.
- Řešení konkrétních problémů
 - Například chybové výpisy, nejasné výsledky testů, validační pravidla, nebo problémy v návrhu. AI nástroj byl používán tak, že mu byl předložen konkrétní vstup a na jeho základě byl navržen rozbor problému a možné další kroky. Sloužil hlavně k přesnému nalezení a vysvětlení problému. Problém sám neopravoval.
- Generování registrace vestavěných metod a regexů
 - Byl vygenerován blok opakujícího se kódu podle jednoho již napsaného vzoru. Vygenerovaný kód byl později značně upraven. AI nástroj nezanášel žádnou logiku ani funkcionalitu.

Kusy kódů které byly generovány nebo například generované regexy jsou v zdrojovém kódu označeny komentářem, který obsahuje i odkaz na konverzaci z které kód vznikl.

7.2 Příložené soubory

Rozsah využití lze lépe dohledat v příloženém souboru `ai-ChatGPT5.4-Thinking-summary.md`. Jedná se o přehledné shrnutí toho, co a jak se v každém chatu řešilo a jak byl AI nástroj využíván. U konverzacích ve kterých došlo ke generování kódu je zde také odkaz na danou konverzaci. Shrnutí bylo vygenerováno ze samotných konverzací - v každé konverzaci spojené s projektem dostal LLM jako prompt následující:

Chci stručné, ale konkrétní shrnutí pouze tohoto jednoho konkrétního chatu.

Důležitá pravidla:

- Vycházej výhradně z obsahu tohoto chatu.
- Nesmíš zasahovat do žádné jiné paměti než do obsahu tohoto konkrétního chatu.
- Nepoužívej paměť, jiné chaty, projektový kontext, domněnky ani externí zdroje.
- Nic si nevymýšlej a nic nedoplňuj z jiných konverzací.
- Pokud nějaká informace v tomto chatu není, napiš výslovně: „V tomto chatu to nebylo řešeno.“
- Nezobecňuj zbytečně; buď co nejkonkrétnější.

Výstup dej v této struktuře:

1. Hlavní téma chatu
2. Co přesně se v chatu řešilo
3. Jak byl asistent v tomto chatu používán

4. Konkrétní výstupy nebo doporučení, které v chatu padly
5. Jednověťé shrnutí

Piš česky.

Samotné podrobné záznamy veškerých konverzací jsou potom v souboru `ai-ChatGPT5.4-Thinking.md`. Včetně záznamu časů kdy konverzace probíhaly.

7.3 Junie

Zde se jednalo o použití integrovaného agenta v IDE *PyCharm*. Jako zdroje měl agent přístup k celému projektu v momentálním stavu dotazu. Veškeré využití tohoto LLM je vyexportováno v příloženém souboru `ai-Junie.pdf`. Ve formě snímku obrazovky s otázkou a poté celé odpovědi od agenta. Tento LLM byl použit pouze k návrhu řešení kruhových importů a k získání návrhu k refaktoringu a kritice OOP modelu. Jelikož agent odpovídal zcestně, návrhy které podal v kódu nejsou zapracované. Konverzace je doložena pouze k zachování transparentnosti.

7.4 Github Copilot GPT 5-mini

K tomuto AI zdroji nejsou v odevzdaném archivu přidány žádné zdroje. Uveden je zde pouze pro transparentnost. Používán byl jako našeptávač kódu a komentářů - veškeré doxygen komentáře jsou generované tímto nástrojem a upravené autorem. Použit byl jako integrovaný nástroj IDE *PyCharm*.

8 Hlavní využití zdroje

- [1] B, S.: Explaining Most Common OOP Design Patterns In the Simplest way possible. Jan 2023. Dostupné z: <https://medium.com/@samyethio891/explaining-most-common-oop-design-pattern-in-the-simplest-way-possible-f7f810179f81>
- [2] Computing, U. R.: Build a Docker Container Image. Dostupné z: <https://chtc.cs.wisc.edu/uw-research-computing/docker-build>
- [3] Design, O. O.: Design Patterns. Dostupné z: <https://www.oodeesign.com/>
- [4] Foundation, P. S.: typing — Support for type hints. Sekce typing.TYPE_CHECKING. Dostupné z: https://docs.python.org/3/library/typing.html#typing.TYPE_CHECKING
- [5] GeeksforGeeks: Abstract Classes in Python. Sep 2025. Dostupné z: <https://www.geeksforgeeks.org/python/abstract-classes-in-python/>
- [6] GitHub: GitHub Copilot - Your AI Pair Programmer. Dostupné z: <https://plugins.jetbrains.com/plugin/17718-github-copilot--your-ai-pair-programmer>
- [7] Inc., D.: Building images. Dostupné z: <https://docs.docker.com/get-started/docker-concepts/building-images/>
- [8] Inc., D.: Writing a Dockerfile. Dostupné z: <https://docs.docker.com/get-started/docker-concepts/building-images/writing-a-dockerfile/>
- [9] Lehtosalo, J.; mypy contributors: mypy 1.20.2 documentation. Dostupné z: <https://mypy.readthedocs.io/en/stable/>

- [10] Macková, M.: Theses Checker. Na stránce uvedena verze 2.2.2. Dostupné z: <https://app.fit.vut.cz/theses-checker>
- [11] Microsoft: The TypeScript Handbook. Apr 2026. Dostupné z: <https://www.typescriptlang.org/docs/handbook/intro.html>
- [12] OpenAI: GPT-5.4 Thinking System Card. Mar 2026. Dostupné z: <https://openai.com/index/gpt-5-4-thinking-system-card/>
- [13] Paradigm, V.: UML Class Diagram Tutorial. Dostupné z: <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-class-diagram-tutorial/>
- [14] Ramos, L. P.: callback. Apr 2025. Dostupné z: <https://realpython.com/ref/glossary/callback/>
- [15] Refactoring.Guru: The Catalog of Design Patterns. Dostupné z: <https://refactoring.guru/design-patterns/catalog>
- [16] W3Schools: Python Tutorial. Dostupné z: <https://www.w3schools.com/python/>
- [17] Zakonov, A.: Meet Junie, Your Coding Agent by JetBrains. Jan 2025. Dostupné z: <https://blog.jetbrains.com/junie/2025/01/meet-junie-your-coding-agent-by-jetbrains/>